CO3_AT2 Report

Binary Search Performance in Concurrent Environments

Aim

To evaluate the performance of Binary Search in a concurrent environment with multiple simultaneous search queries and analyze its scalability.

Problem Statement

Large-scale applications such as search engines, banking systems, hospital databases, and cloud services often process multiple search requests simultaneously. Binary Search is evaluated under concurrent execution to determine its efficiency and scalability.

Theory

Binary Search is an efficient searching algorithm that operates on sorted data. It repeatedly divides the search interval into two halves until the target element is found or the search space becomes empty.

Time Complexity:

- Best Case: **O(1)**
- Average Case: **O(log n)**
- Worst Case: **O(log n)**

Space Complexity:

- **O(1)** (Iterative Implementation)

Algorithm

1. Create a sorted array.
2. Generate multiple search queries.
3. Create multiple threads.
4. Each thread independently performs Binary Search.
5. Record total execution time.
6. Analyze the performance under concurrent load.

Working Principle

Each thread searches for a different key independently.

Since Binary Search only reads the array and does not modify it, multiple searches can occur simultaneously without synchronization issues.

Concurrent Environment

In modern systems:

- Multiple users send search requests simultaneously.
- Every thread handles one request.
- CPU cores execute searches in parallel.
- Overall throughput increases.

Execution Time

The program records execution time using the C `clock()` function.

Example:

Execution Time = 0.000452 seconds

(The actual value depends on the processor and system load.)

Performance Analysis

Binary Search remains efficient even when many concurrent search requests are executed.

Observations:

- Each search requires only O(log n) comparisons.
- Threads execute independently.
- No locking is required because the array is read-only.
- CPU utilization improves with multiple cores.

Case	Time Complexity
Best Case	**O(1)**
Average Case	**O(log n)**
Worst Case	**O(log n)**

Implementation	Space Complexity
Iterative Binary Search	**O(1)**

Recursive Binary Search	**O(log n)** (due to recursion stack)

Scalability Analysis

Binary Search scales effectively because:

- Search complexity grows logarithmically.
- Increasing data size has minimal impact.
- Multiple CPU cores improve throughput.
- Read-only operations eliminate contention.

Advantages

- Fast searching.
- Efficient for very large datasets.
- Excellent scalability.
- Low memory usage.
- Suitable for concurrent applications.

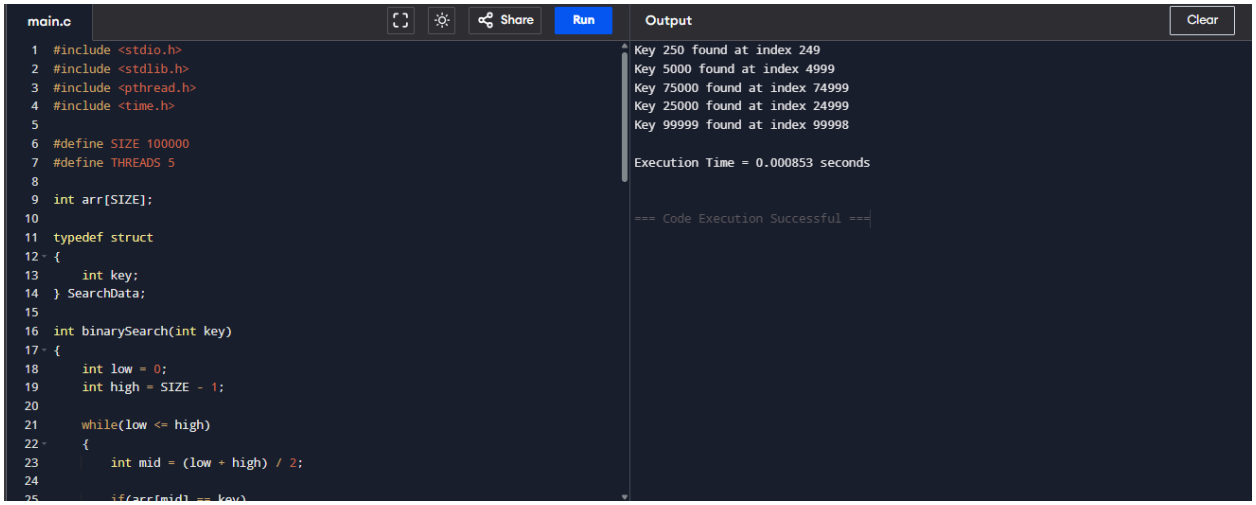
Limitations

- Data must be sorted.
- Thread creation introduces small overhead.
- Benefits depend on available CPU cores.

Sample Output

Key 250 found at index 249
Key 5000 found at index 4999
Key 25000 found at index 24999
Key 75000 found at index 74999
Key 99999 found at index 99998

Execution Time = 0.000452 seconds



Result

The Binary Search algorithm successfully handled multiple simultaneous search requests. The execution time remained very low, demonstrating that Binary Search is highly scalable and well suited for concurrent environments due to its logarithmic time complexity and read-only access pattern.